Complete programs for the affine constructions

Human Compression Project

The six complete programs below are part of the proof. They identify both affine translation kernels, both actual presentation kernels, the exceptional seed and sign witness, and the complete order-384 nonexistence search. Diagnostic checks inside a required file remain included. The operational driver and scoped Lean files are inventoried separately.

phase250_affine_check.py

```
"""Phase250 {6,4} affine exploration; arithmetic reuses phase2 definitions."""
from collections import deque
from itertools import combinations
from pathlib import Path
import hashlib
import json
import math
import argparse

ROOT = Path(__file__).resolve().parent
I = ((1, 0, 0), (0, 1, 0), (0, 0, 1))
ZERO = (0, 0, 0)
ONE = (I, ZERO)


def mm(a, b):
    return tuple(tuple(sum(a[i][k] * b[k][j] for k in range(3))
                       for j in range(3)) for i in range(3))


def mv(a, v):
    return tuple(sum(a[i][j] * v[j] for j in range(3)) for i in range(3))


def mul(x, y):
    return mm(x[0], y[0]), tuple(a + b for a, b in zip(mv(x[0], y[1]), x[1]))


def inverse(x):
    m = tuple(zip(*x[0]))
    return m, tuple(-a for a in mv(m, x[1]))


def power(x, n):
    y = ONE
    for _ in range(n):
        y = mul(y, x)
    return y


def word(*xs):
    y = ONE
    for x in xs:
        y = mul(y, x)
    return y


def conjugate(x, y):
    return word(inverse(y), x, y)
```

```
def determinant(a, b, c):
    return (a[0] * (b[1] * c[2] - b[2] * c[1])
            - a[1] * (b[0] * c[2] - b[2] * c[0])
            + a[2] * (b[0] * c[1] - b[1] * c[0]))


def subgroup(gens):
    result = {ONE}
    queue = deque(result)
    while queue:
        x = queue.popleft()
        for g in gens:
            y = mul(g, x)
            if y not in result:
                if len(result)>96: raise RuntimeError("PHASE250_REJECT finite parabolic
                 ↪  bound")
                result.add(y)
                queue.append(y)
    return result


def need(ok, reason):
    if not ok: raise RuntimeError("PHASE250_REJECT "+reason)
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--out',type=Path,required=True)
    ap.add_argument('--corrupt-translation',action='store_true');args=ap.parse_args()
    need(not args.out.exists() and not args.out.is_symlink(),'fresh output')
    a=(((0,1,0),(1,0,0),(0,0,1)),ZERO)
    b=(((-1,0,0),(0,0,1),(0,1,0)),ZERO if args.corrupt_translation else (1,0,0))
    c=(((1,0,0),(0,1,0),(0,0,-1)),ZERO);gens=[a,b,c]
    comm=lambda x,y:word(inverse(x),inverse(y),x,y)
    relators=[power(x,2) for x in gens]+[power(word(a,b),6),power(word(b,c),4),power(word(a
     ↪  ,c),2),power(comm(a,power(word(b,c),2)),2)]
    need(all(x==ONE for x in relators),'all literal universal relators')
    signed=[(1,0,0),(0,1,0),(0,0,1),(-1,0,0),(0,-1,0),(0,0,-1)]
    permutations=[[signed.index(mv(g[0],v))+1 for v in signed] for g in gens]
    need(permutations==[[2,1,3,5,4,6],[4,3,2,1,6,5],[1,2,6,4,5,3]],'literal point quotient')
    lifts={I:ONE};words={I:''};queue=deque([I])
    while queue:
        matrix=queue.popleft()
        for label,gen in zip('abc',gens):
            y=mul(gen,lifts[matrix])
            if y[0] not in
             ↪  lifts:lifts[y[0]]=y;words[y[0]]=label+words[matrix];queue.append(y[0])
    need(len(lifts)==48,'point group48')
    transitions=[]
    for matrix,lift in lifts.items():
        for label,gen in zip('abc',gens):
            y=mul(gen,lift);z=mul(inverse(lifts[y[0]]),y);need(z[0]==I,'transition
             ↪  translation')
            transitions.append(dict(word='('+words[y[0]]+')^-1'+label+words[matrix],vector=z
             ↪  [1]))
    vectors=sorted(set(tuple(x['vector']) for x in transitions));index=0
    for basis in combinations(vectors,3):index=math.gcd(index,abs(determinant(*basis)))
    need(index==4,'translation lattice index4')
    need(all((v[0]-v[1])%2==0 and (v[1]-v[2])%2==0 for v in vectors),'equal parity
     ↪  containment')
    need(all(v in vectors for v in [(-2,0,0),(0,0,2),(-1,-1,-1),(-1,1,-1)]),'explicit full
     ↪  lattice generators')
    source=[
        [power(word(b,conjugate(c,word(b,a))),2),power(word(conjugate(c,b),conjugate(b,a)),
         ↪  2),power(word(c,conjugate(b,word(a,b))),2)],
        [power(word(conjugate(a,b),conjugate(b,word(a,c))),2),power(word(conjugate(b,word(a
         ↪  ,c,b)),a),2),power(word(conjugate(a,word(b,c,b)),conjugate(b,word(a,b))),2)],
        [power(word(b,a,c),6),power(word(c,b,a),6),power(conjugate(word(c,b,a),b),6)]]
    need(all(w[0]==I for row in source for w in row),'source words are translations')
    bases=[[w[1] for w in row] for row in source]
    expected=[[(2,0,0),(0,-2,0),(0,0,-2)],[(2,0,2),(2,-2,0),(-2,-2,0)],[(2,-2,2),(2,-2,-2),
     ↪  (-2,-2,-2)]]
    need(bases==expected,'exact source vector triples')
    dets=[abs(determinant(*row)) for row in bases];need(dets==[8,16,32],'source
     ↪  determinants')
    contains=[lambda v:all(x%2==0 for x in v),lambda v:all(x%2==0 for x in v) and sum(v)%4==0,
```

```
        lambda v:all(x%2==0 for x in v) and (v[0]-v[1])%4==0 and (v[1]-v[2])%4==0]
    for base,test in zip(bases,contains):
        need(all(test(mv(q,v)) for q in lifts for v in base),'period invariance under every
         ↪  point element')
    left=subgroup([a,b]);right=subgroup([b,c]);need(len(left)==12 and
     ↪  len(right)==8,'adjacent orders')
    need(len({g[0] for g in left})==12 and len({g[0] for g in right})==8,'faithful linear
     ↪  parabolics')
    overlaps=[dict(matrix=x[0],left_translation=x[1],right_translation=y[1],difference=tupl
     ↪  e(s-t for s,t in zip(x[1],y[1]))) for x in sorted(left) for y in sorted(right) if
     ↪  x[0]==y[0]]
    need(len(overlaps)==2 and all(row['difference']==ZERO for row in overlaps),'exact
     ↪  two-element overlap')
    need(left&right=={ONE,b},'middle involution intersection')
    # A raw-difference trap: raw transitions span only e1, whereas the
    # correct normalized transitions span e1,e2. This tests the lemma's orientation.
    neg=(((-1,0,0),(0,-1,0),(0,0,1)),ZERO)
    shift=(neg[0],(1,0,0));swap=a
    pure={I:ONE};todo=deque([I])
    while todo:
        matrix=todo.popleft()
        for gen in [neg,swap]:
            y=mul(gen,pure[matrix])
            if y[0] not in pure:pure[y[0]]=y;todo.append(y[0])
    raw=[];normalized=[]
    for lift in pure.values():
        for gen in [shift,neg,swap]:
            y=mul(gen,lift)
            raw.append(mul(y,inverse(pure[y[0]]))[1])
            normalized.append(mul(inverse(pure[y[0]]),y)[1])
    need(set(raw)=={ZERO,(1,0,0)},'raw differences only one coordinate')
    need(all(v[2]==0 for v in normalized) and any(v[1]!=0 for v in normalized),'normalized
     ↪  transitions recover second coordinate')
    minor_gcd=0
    for v,w in
     ↪  combinations(normalized,2):minor_gcd=math.gcd(minor_gcd,abs(v[0]*w[1]-v[1]*w[0]))
    need(minor_gcd==1,'normalized full planar lattice')
    print('SCHREIER_NORMALIZATION_CONTROL_PASS raw_rank1 normalized_rank2')
    out=dict(status='PASS',generators=gens,point_permutations=permutations,point_group_orde
     ↪  r=48,
        normalization_control=dict(raw_rank=1,normalized_rank=2),translation_lattice='2Z^3+
         ↪  Z(1,1,1)',translation_index=4,schreier_transitions=transitions,
        source_word_vectors=bases,period_determinants=dets,period_indices=[2,4,8],
        finite_order_constants=[96,192,384],adjacent_orders=[12,8],overlaps=overlaps,
        uniform_scope='Every positive m: lattice mLambda, exact order constant*m^3, same
         ↪  type6,4 and IP.',
        retained_step='Exact abstract point-kernel presentation is separately verified in
         ↪  GAP.',
        arithmetic_provenance='Elementary matrix functions reused from
         ↪  phase2_cubic_lattice_check.py; no old top-level code executed.')
    args.out.write_text(json.dumps(out,indent=2)+'\n')
    print('PHASE250_AFFINE_PASS point48 lattice_index4 period_indices2,4,8 overlap2')
if __name__=='__main__':main()
```

phase250_affine64_kernel.g

```
F:=FreeGroup("a","b","c");;a:=F.1;;b:=F.2;;c:=F.3;;
U:=F/[a^2,b^2,c^2,(a*b)^6,(b*c)^4,(a*c)^2,Comm(a,(b*c)^2)^2];;
ug:=GeneratorsOfGroup(U);;
qg:=[(1,2)(4,5),(1,4)(2,3)(5,6),(3,6)];;Q:=Group(qg);;
if Size(Q)<>48 then Error("point group");fi;
phi:=GroupHomomorphismByImages(U,Q,ug,qg);;
if phi=fail or not IsSurjective(phi) then Error("point epimorphism");fi;
K:=Kernel(phi);;P:=Image(IsomorphismFpGroup(K));;S:=SimplifiedFpGroup(P);;
Print("INDEX ",Index(U,K)," RAW GENERATORS ",Length(GeneratorsOfGroup(P))," RAW RELATORS
 ↪  ",Length(RelatorsOfFpGroup(P)),"\n");
Print("SIMPLIFIED GENERATORS ",GeneratorsOfGroup(S)," RELATORS ",RelatorsOfFpGroup(S),"\n");
if Length(GeneratorsOfGroup(S))<>3 or not IsAbelian(S) or AbelianInvariants(S)<>[0,0,0] then
 ↪  Error("notZ3");fi;
Print("AFFINE64_Z3_KERNEL_PASS\n");
QUIT_GAP(0);
```

phase250_fixed_groups.g

```
Check250:=function(ok,s) if not ok then Error(Concatenation("P250_REJECT ",s));fi;end;;
IP250:=function(rr)
 local subsets,hs,i,j;
 subsets:=Combinations([1..3]);hs:=List(subsets,x->Group(rr{x},One(rr[1])));
 for i in [1..8] do for j in [1..8] do
 if Intersection(hs[i],hs[j])<>hs[Position(subsets,Intersection(subsets[i],subsets[j]))]
  ↪  then return false;fi;
 od;od;return true;
end;;
Run250:=function()
 local F,a,b,c,x,w,s,U,iso,G,rr,D,dd,Q,qi,qp,qg,ep,x1,y1,z1,corrected,rels;
F:=FreeGroup("a","b","c");;a:=F.1;;b:=F.2;;c:=F.3;;x:=b*c;;
w:=b*a*c*b*a*(c*b)^2*a;;
for s in [2,3,4] do
 U:=F/[a^2,b^2,c^2,(a*b)^6,x^(2^s),(a*c)^2,Comm(a,x^4),w^2];;
 iso:=IsomorphismPermGroup(U);;G:=Image(iso);;rr:=List(GeneratorsOfGroup(U),g->Image(iso,g)
  ↪  );;
 Check250(Size(G)=384*2^(s-2) and Order(rr[1]*rr[2])=6 and Order(rr[2]*rr[3])=2^s and
  ↪  IP250(rr),"exceptional family");
 # Use a literal dihedral presentation to avoid generator-convention assumptions.
 Q:=F/[a,b^2,c^2,x^(2^s)];;qi:=IsomorphismPermGroup(Q);;qp:=Image(qi);;qg:=List(GeneratorsO
  ↪  fGroup(Q),g->Image(qi,g));;
 ep:=GroupHomomorphismByImages(G,qp,rr,qg);;Check250(ep<>fail and IsSurjective(ep),"pair
  ↪  retraction");
 Print("EXCEPTIONAL64 s ",s," ORDER ",Size(G)," IP64 RETRACTION PASS\n");
od;
x1:=(b*c^(b*a))^2;;y1:=(c^b*b^a)^2;;z1:=(c*b^(a*b))^2;;

for corrected in [false,true] do
 rels:=[a^2,b^2,c^2,(a*b)^6,x^16,(a*c)^2,Comm(a,x^2)^2,Comm(a,x^4),y1^2*x^-4,z1^2*x^4];
 if corrected then Add(rels,x1^2*x^4);else Add(rels,x1^2*x^-4);fi;
 U:=F/rels;iso:=IsomorphismPermGroup(U);G:=Image(iso);rr:=List(GeneratorsOfGroup(U),g->Imag
  ↪  e(iso,g));
 if corrected then
  Check250(Size(G)=3072 and Order(rr[2]*rr[3])=16 and IP250(rr),"corrected source sign");
  Print("CORRECTED45 n10 s4 ORDER3072 ROTATION16 IP64 PASS\n");
 else
  Check250(Size(G)=1536 and Order(rr[2]*rr[3])=8,"literal source sign discrepancy");
  Print("LITERAL45 n10 s4 ORDER1536 ROTATION8 REJECTED_TARGET\n");
 fi;
od;

a:=(1,2);x:=(1,2,3);b:=x^2;
Check250(a^-1*b*a=b^-1 and b^2=x^4 and x^8<>One(Group(a,x)),"commutation false weakening");
Print("PERIOD_COMMUTATION_FALSE_WEAKENING_REJECTED\nPHASE250_FIXED_GROUPS_PASS\n");


end;;
Run250();
QUIT_GAP(0);
```

phase250_base384_census.g

```
# Complete marked search; SmallGroups catalogue completeness is retained.
Check250:=function(ok,s) if not ok then Error(Concatenation("P250_REJECT ",s));fi;end;;
Search250:=function(G,k)
 local classes,cls,c,C,inv,b,orbit,a,pairs,triples,ip;
 pairs:=0;triples:=0;ip:=0;
 classes:=Filtered(ConjugacyClasses(G),cl->Order(Representative(cl))=2);
 for cls in classes do
  c:=Representative(cls);C:=Centralizer(G,c);
  inv:=Filtered(Elements(G),g->Order(g)=2 and Order(g*c)=k);
  while Length(inv)>0 do
   b:=inv[1];orbit:=Orbit(C,b);inv:=Difference(inv,orbit);pairs:=pairs+1;
   for a in Filtered(Elements(C),g->Order(g)=2 and Order(g*b)=6) do
    if Size(Group(a,b,c))=Size(G) then
     triples:=triples+1;
     if Size(Intersection(Group(a,b),Group(b,c)))=2 then ip:=ip+1;fi;
    fi;
```

```
   od;
  od;
 od;
 return [pairs,triples,ip];
end;;
Run250:=function()
 local N,eligible,pairs,triples,id,G,stats,positive;
 Check250(LoadPackage("smallgrp")=true,"SmallGroups package");
 N:=NumberSmallGroups(384);Check250(N=20169,"complete order384 count");
 eligible:=0;pairs:=0;triples:=0;
 Print("SMALLGRP_VERSION ",PackageInfo("smallgrp")[1].Version," GROUP_COUNT ",N,"\n");
 for id in [1..N] do
  G:=SmallGroup(384,id);
  if Exponent(G) mod 16=0 then
   eligible:=eligible+1;stats:=Search250(G,16);
   pairs:=pairs+stats[1];triples:=triples+stats[2];
   Check250(stats[3]=0,"unexpected Cgroup of type6,16");
  fi;
 od;
 Check250(eligible=1178 and pairs=4436 and triples=0,"bound exhaustive counts");
 Print("NO_CGROUP_6_16_ORDER384_PASS GROUPS ",N," ELIGIBLE ",eligible," PAIRS ",pairs,"
  ↪  GENERATING ",triples,"\n");
 positive:=Search250(SmallGroup(384,18046),4);
 Check250(positive[3]>0,"same search detects the known6,4 seed");
 Print("POSITIVE_CGROUP_6_4_CONTROL ",positive,"\nPHASE250_CENSUS_PASS\n");
end;;
Run250();
QUIT_GAP(0);
```

phase2_cubic_lattice_check.py

```
"""Exact integer affine certificate for the three {6,6} lattice families."""
from collections import deque
from itertools import combinations
from pathlib import Path
import hashlib
import json
import math

ROOT = Path(__file__).resolve().parent
I = ((1, 0, 0), (0, 1, 0), (0, 0, 1))
ZERO = (0, 0, 0)
ONE = (I, ZERO)


def mm(a, b):
    return tuple(tuple(sum(a[i][k] * b[k][j] for k in range(3))
                       for j in range(3)) for i in range(3))


def mv(a, v):
    return tuple(sum(a[i][j] * v[j] for j in range(3)) for i in range(3))


def mul(x, y):
    return mm(x[0], y[0]), tuple(a + b for a, b in zip(mv(x[0], y[1]), x[1]))


def inverse(x):
    m = tuple(zip(*x[0]))
    return m, tuple(-a for a in mv(m, x[1]))


def power(x, n):
    y = ONE
    for _ in range(n):
        y = mul(y, x)
    return y


def word(*xs):
    y = ONE
```

```
    for x in xs:
        y = mul(y, x)
    return y


def conjugate(x, y):
    return word(inverse(y), x, y)


def determinant(a, b, c):
    return (a[0] * (b[1] * c[2] - b[2] * c[1])
            - a[1] * (b[0] * c[2] - b[2] * c[0])
            + a[2] * (b[0] * c[1] - b[1] * c[0]))


def subgroup(gens):
    result = {ONE}
    queue = deque(result)
    while queue:
        x = queue.popleft()
        for g in gens:
            y = mul(g, x)
            if y not in result:
                result.add(y)
                queue.append(y)
    return result


a = (((0, 1, 0), (1, 0, 0), (0, 0, 1)), ZERO)
b = (((-1, 0, 0), (0, 0, 1), (0, 1, 0)), (1, 0, 0))
c = (((0, -1, 0), (-1, 0, 0), (0, 0, 1)), ZERO)
gens = [a, b, c]
signed_basis = [(1, 0, 0), (0, 1, 0), (0, 0, 1),
                (-1, 0, 0), (0, -1, 0), (0, 0, -1)]
point_permutations = [[signed_basis.index(mv(g[0], v)) + 1 for v in signed_basis]
                      for g in gens]
assert point_permutations == [[2, 1, 3, 5, 4, 6],
                              [4, 3, 2, 1, 6, 5],
                              [5, 4, 3, 2, 1, 6]]
assert all(power(x, 2) == ONE for x in gens)
assert power(word(a, b), 6) == power(word(b, c), 6) == ONE
assert power(word(a, c), 2) == power(word(c, b, a, b), 3) == ONE

# Schreier representatives for the 48 possible signed linear parts.
lifts = {I: ONE}
lift_words = {I: ""}
queue = deque([I])
while queue:
    matrix = queue.popleft()
    for label, generator in zip("abc", gens):
        y = mul(generator, lifts[matrix])
        if y[0] not in lifts:
            lifts[y[0]] = y
            lift_words[y[0]] = label + lift_words[matrix]
            queue.append(y[0])
assert len(lifts) == 48
schreier = []
for matrix, lift in lifts.items():
    for label, generator in zip("abc", gens):
        y = mul(generator, lift)
        # y and the representative for its linear part differ by this translation.
        z = mul(y, inverse(lifts[y[0]]))
        assert z[0] == I
        if z[1] != ZERO:
            schreier.append({"vector": z[1],
                             "word": f"{label}{lift_words[matrix]}({lift_words[y[0]]})^-1"})
vectors = sorted({tuple(x["vector"]) for x in schreier})
assert all(all(v % 2 == 0 for v in row) and sum(row) % 4 == 0 for row in vectors)
index = 0
for basis in combinations(vectors, 3):
    index = math.gcd(index, abs(determinant(*basis)))
assert index == 16  # exactly 2D3, not just a sublattice contained in it

pair_left = subgroup([a, b])
```

```
pair_right = subgroup([b, c])
assert len(pair_left) == len(pair_right) == 12
assert len({x[0] for x in pair_left}) == len({x[0] for x in pair_right}) == 12
overlaps = []
for x in sorted(pair_left):
    for y in sorted(pair_right):
        if x[0] == y[0]:
            difference = tuple(u - v for u, v in zip(x[1], y[1]))
            overlaps.append({"linear": x[0], "left_translation": x[1],
                             "right_translation": y[1], "difference": difference})
assert len(overlaps) == 4
assert sorted(x["difference"] for x in overlaps) == [ZERO, ZERO, (0, 2, 2), (0, 2, 2)]
assert pair_left & pair_right == {ONE, b}

source_bases = [
    [conjugate(power(word(a, c, b), 4), b), power(word(c, b, a), 4),
     conjugate(power(word(c, b, a), 4), b)],
    [conjugate(power(word(power(word(b, c), 3), power(word(a, b), 3)), 2), word(a, b)),
     conjugate(power(word(power(word(b, c), 3), power(word(a, b), 3)), 2), c),
     conjugate(power(word(power(word(b, c), 3), power(word(a, b), 3)), 2), word(c, b))],
    [conjugate(power(word(power(word(a, b), 2), power(word(c, b), 2)), 3), a),
     power(word(power(word(b, c), 2), power(word(b, a), 2)), 3),
     conjugate(power(word(power(word(b, a), 2), power(word(b, c), 2)), 3), c)],
]
assert all(x[0] == I for row in source_bases for x in row)
bases = [[x[1] for x in row] for row in source_bases]
indices_in_z3 = [abs(determinant(*basis)) for basis in bases]
assert indices_in_z3 == [64, 128, 256]
assert all(all(v % 4 == 0 for v in row) for basis in bases for row in basis)
orders = [48 * index // 16 for index in indices_in_z3]
assert orders == [192, 384, 768]

data = {
    "status": "PASS",
    "arithmetic": "integer affine transformations only; no floating point",
    "generators": gens,
    "signed_basis_order": signed_basis,
    "point_generator_permutations_as_image_lists": point_permutations,
    "permutation_convention": "GAP right action: p maps to inverse of its column-action
     ↪  matrix; generators are involutions, so their matrices agree exactly",
    "point_group_order": len(lifts),
    "translation_lattice": "2D3={(2x,2y,2z):x+y+z even}",
    "translation_index_in_Z3": index,
    "schreier_vectors": vectors,
    "schreier_word_certificates": schreier,
    "parabolic_overlap_certificate": overlaps,
    "source_word_translation_bases": bases,
    "lattice_indices_in_Z3": indices_in_z3,
    "orders_for_m_one": orders,
    "all_m_formula": ["192*m^3", "384*m^3", "768*m^3"],
    "input_hashes": {f: hashlib.sha256((ROOT / f).read_bytes()).hexdigest()
                     for f in [Path(__file__).name, "phase2_cubic_kernel_check.g"]},
}
(ROOT / "phase2_cubic_lattice_check.json").write_text(json.dumps(data, indent=2) + "\n")
print("PASS: point group48, kernel2D3 index16; source lattices64,128,256 in Z3")
print("PASS: pair parabolics12 each; only extra linear overlaps differ by (0,2,2)")
print("PASS: exact source-word lattices yield 192*m^3,384*m^3,768*m^3 for every m>=1")
```

phase2_cubic_kernel_check.g

```
LogTo("research/leemans/phase2_cubic_kernel_check.log");;
F:=FreeGroup("a","b","c");;
a:=F.1;; b:=F.2;; c:=F.3;;
U:=F/[a^2,b^2,c^2,(a*b)^6,(b*c)^6,(a*c)^2,(c*b*a*b)^3];;
ug:=GeneratorsOfGroup(U);;
Q:=Group((1,2)(4,5),(1,4)(2,3)(5,6),(1,5)(2,4));;
qg:=[(1,2)(4,5),(1,4)(2,3)(5,6),(1,5)(2,4)];;
if Size(Q)<>48 then Error("Wrong point group"); fi;
phi:=GroupHomomorphismByImages(U,Q,ug,qg);;
if phi=fail or not IsSurjective(phi) then Error("No point quotient"); fi;
Print("POINT QUOTIENT order=",Size(Q)," epimorphism=true\n");
K:=Kernel(phi);;
```

```
Print("KERNEL index=",Index(U,K)," Schreier generators=",Length(GeneratorsOfGroup(K)),"\n");
Print("KERNEL WORD GENERATORS=",GeneratorsOfGroup(K),"\n");
iso:=IsomorphismFpGroup(K);;
P:=Image(iso);;
Print("RAW KERNEL PRESENTATION generators=",Length(GeneratorsOfGroup(P)),
      " relators=",Length(RelatorsOfFpGroup(P)),"\n");
Print("RAW RELATORS=",RelatorsOfFpGroup(P),"\n");
S:=SimplifiedFpGroup(P);;
Print("SIMPLIFIED generators=",GeneratorsOfGroup(S)," relators=",RelatorsOfFpGroup(S),"\n");
if Length(GeneratorsOfGroup(S))<>3 then Error("Unexpected kernel rank"); fi;
# IsAbelian for this explicit small presentation is an exact proof, not an
# inference from abelianization. The separate affine certificate proves its
# abelianization has rank three, hence no omitted torsion relations.
if not IsAbelian(S) then Error("Kernel presentation is not abelian"); fi;
if AbelianInvariants(S)<>[0,0,0] then Error("Kernel is not free abelian rank3"); fi;
Print("KERNEL IS Z^3: exact Reidemeister-Schreier/Tietze presentation PASS\n");
LogTo();
QUIT;
```